MODULE 06 · DOMAIN 1 - SDLC AUTOMATION

CodeDeploy on EC2 and On-Premises

The agent, the appspec and its hooks, deployment configurations, Auto Scaling, blue/green on EC2, and rolling back.

DOMAIN 1 of 6 EXAM WEIGHT 22% MODULE 6 of 34

admin@sthithapragna.com

CodeDeploy on servers is a **script runner** with a **traffic switch**. You write the scripts; it decides when, where, and how many at a time.

The moving parts

PART	WHAT IT IS
Application	A name that groups everything for one piece of software
Deployment group	Targets, config, load balancer, alarms, rollback settings
Deployment configuration	How many instances may be out of service at once
Revision	A zip or tar bundle in S3, or a GitHub commit, with an appspec
Agent	Software on each instance that polls CodeDeploy and runs the scripts

ON THE EXAM

Targets are chosen by EC2 tags, on-premises tags, or Auto Scaling group names in the deployment group.

The agent, and the two identities

- The **agent** polls CodeDeploy for work. No inbound ports are opened on the instance.
- The **instance profile** lets the agent download the revision from S3.
- The **service role** lets CodeDeploy call Auto Scaling and Elastic Load Balancing for you.
- On-premises servers are registered with an IAM identity and tagged, then run the same agent.

> ON THE EXAM
>
> Install and update the agent fleet-wide with Systems Manager, rather than baking scripts into every launch.

An appspec for a server

```yaml
version: 0.0
os: linux
files:
  - source: /build
    destination: /var/www/app
hooks:
  ApplicationStop:
    - location: scripts/stop.sh
  BeforeInstall:
    - location: scripts/clear-cache.sh
  ValidateService:
    - location: scripts/health-check.sh
```

9 Runs before files are copied: the place to clear a cache.

11 A non-zero exit fails this instance's deployment.

ON THE EXAM

The appspec lives at the root of the revision bundle. Its hooks are just scripts, judged by their exit codes.

Lifecycle events, in order, with a load balancer

EVENT	SCRIPT FROM	TYPICAL USE
BeforeBlockTraffic, BlockTraffic, AfterBlockTraffic	Previous revision	Drain from the load balancer
ApplicationStop	Previous revision	Stop the old version gracefully
DownloadBundle, BeforeInstall	New revision	Fetch; clear old files, back up
Install, AfterInstall	New revision	Copy files; configure, set permissions
ApplicationStart, ValidateService	New revision	Start; health check
BeforeAllowTraffic, AllowTraffic, AfterAllowTraffic	New revision	Warm up; rejoin; confirm

ON THE EXAM

DownloadBundle, Install, BlockTraffic and AllowTraffic are CodeDeploy's own steps. You cannot attach scripts to them.

The previous-revision trap

- ApplicationStop, BeforeBlockTraffic and AfterBlockTraffic run scripts from the **last successful** revision.
- So a broken stop script, once deployed, breaks **every later deployment**, including the fix.
- Fixing the script in the new revision does not help: its stop script is not the one that runs.
- Deploy with **ignoreApplicationStopFailures**, and the fixed revision becomes the new last successful one.

> ON THE EXAM
>
> Every deployment now fails at ApplicationStop, even the fix: ignore application stop failures for one deployment.

Three more deployment settings

- **fileExistsBehavior**: what to do when a file exists that the last deployment did not put there.
- DISALLOW, the default, fails the deployment. OVERWRITE replaces the file. RETAIN keeps it.
- Each hook script has a **timeout**, one hour at most, set in the appspec.
- **runas** sets which operating system user runs a script.

> ON THE EXAM
>
> Deployment fails because files already exist, often after a manual hotfix: set the file-exists behaviour to OVERWRITE.

Deployment configurations

CONFIGURATION	HOW MANY AT ONCE	TRADE-OFF
`AllAtOnce`	Every instance together	Fastest; an outage if it fails
`HalfAtATime`	Up to half the fleet	Middle ground; half your capacity gone
`OneAtATime`	One instance	Safest; slowest on a large fleet
`Custom: HOST_COUNT`	Keep at least N instances healthy	Exact numbers
`Custom: FLEET_PERCENT`	Keep at least N% healthy	Scales with fleet size

ON THE EXAM

Minimum healthy hosts is what must stay up. The rest is how many CodeDeploy may take down at once.

Doing the arithmetic

- 20 instances, FLEET_PERCENT 75: at least 15 stay healthy, so up to **5** update at once.
- 20 instances, HalfAtATime: 10 stay healthy, and the other 10 carry no traffic.
- 'At least 75% available, as fast as possible' is FLEET_PERCENT 75, not 25.
- OneAtATime also keeps 75% available, but takes twenty rounds instead of four.

> ON THE EXAM
>
> Read the percentage as what stays up. A distractor offering 25% is counting the ones that go down.

Zonal deployments

- A **zonal configuration** deploys to one Availability Zone at a time. EC2 and on-premises only.
- **Monitor duration** waits after each zone before starting the next.
- **First zone monitor duration** can set a longer bake for the first zone.
- **Minimum healthy hosts per zone** works alongside the fleet-wide minimum.

> ON THE EXAM
>
> 'One Availability Zone at a time, with a bake period before the next': a zonal configuration, not separate deployment groups.

In-place or blue/green, on EC2?

BLUE/GREEN

A new fleet, then switch traffic

- Copies the Auto Scaling group, or uses instances you provide
- Full capacity throughout
- Rollback moves traffic back: fast
- Old fleet terminated after a wait you set

IN-PLACE

Update the instances you have

- No extra instances, no extra cost
- Capacity reduced while batches update
- Rollback is another full deployment: slow
- Works on-premises; blue/green does not

THE TIEBREAKER

Fastest rollback, full capacity: blue/green. On-premises servers, or no spare capacity: in-place.

CodeDeploy and Auto Scaling

- A **launch lifecycle hook** holds each new instance until CodeDeploy installs the last successful revision.
- Scale-out **during** a deployment gets the older revision, then an automatic **follow-on deployment** updates it.
- If new instances keep failing their deployment, they are terminated and replaced, in a loop.
- **Termination hooks** run your stop scripts before scale-in terminates an instance.

> ON THE EXAM
>
> Graceful shutdown on scale-in, using the existing CodeDeploy setup: enable termination hooks on the deployment group.

Rolling back

- Automatic rollback triggers on a **failed deployment**, on an **alarm**, or both.
- Alarms are attached to the deployment group; a firing alarm stops the deployment.
- A rollback is a **new deployment** of the last good revision, with a new deployment ID.
- A single deployment can **override** the group's alarm configuration.

> ON THE EXAM
>
> Roll back automatically when application errors rise: an alarm on the deployment group plus rollback on alarm.

Watching deployments

- **Triggers** send deployment and instance events to SNS. EventBridge receives state changes too.
- The agent writes a deployment log on each instance with every script's output.
- Ship those logs to CloudWatch Logs with the CloudWatch agent, and query them with Logs Insights.
- Connecting to each failed instance to read its log does not scale.

> ON THE EXAM
>
> 'Find which lifecycle event failed across many instances, efficiently': agent logs in CloudWatch, queried with Logs Insights.

> Revision 41 of an application deployed successfully, but its stop.sh script references a file that revision 41 also removed. Revision 42 corrects stop.sh.
>
> Every deployment of revision 42 fails at the ApplicationStop event on every instance, and so does an attempt to redeploy revision 40.

How should the DevOps engineer get revision 42 deployed?

A. Edit stop.sh in revision 42 so that it exits 0 when the file is missing

B. Create a deployment of revision 42 with ignoreApplicationStopFailures set to true

C. Reinstall the CodeDeploy agent on every instance

D. Change the deployment configuration to OneAtATime

◆ PAUSE AND DECIDE

How should the DevOps engineer get revision 42 deployed?

A Fix stop.sh in revision 42

The trap. ApplicationStop runs revision 41's script, the last successful one. Revision 42's is never used here.

B ignoreApplicationStopFailures

Skips the broken stop step once. Revision 42 then becomes the last successful revision.

C Reinstall the agent

The agent is working correctly. It is running exactly the script it is meant to run.

D OneAtATime

Changes how many fail at once, not whether they fail.

THE SIGNAL

Even a redeploy of an older revision fails at ApplicationStop: the last successful revision's stop script is broken.

> A company runs 18 EC2 instances across three Availability Zones. After a release once failed in every zone at the same time, it wants deployments to go one Availability Zone at a time.
>
> The first zone must bake for 60 minutes before the next zone starts, and each zone must keep at least 2 healthy instances throughout.

Which solution meets these requirements with the LEAST operational overhead?

A. A custom deployment configuration with a zonal configuration: first-zone monitor duration 60 minutes, minimum 2 healthy hosts per zone

B. The CodeDeployDefault.OneAtATime deployment configuration

C. Three deployment groups, one per zone, run in sequence by the pipeline with a wait action between them

D. The HalfAtATime deployment configuration with a CloudWatch alarm for rollback

◆ PAUSE AND DECIDE

Which solution meets these requirements with the LEAST operational overhead?

A Zonal configuration

Zone by zone, first-zone bake, per-zone minimum: every requirement is a field in one configuration.

B OneAtATime

Ignores zones entirely, and has no bake period between them.

C Three deployment groups

The workaround from before zonal configuration. Works, but triples groups and pipeline logic.

D HalfAtATime plus alarm

Half the fleet can span every zone at once: the exact failure the company wants to avoid.

THE SIGNAL

Zone at a time, bake between zones: zonal configuration. Separate groups per zone is the old workaround.

A worker application runs on an Auto Scaling group whose instances are deployed with CodeDeploy. The application processes long-running jobs.

When the group scales in, instances are terminated while jobs are still running, and the jobs are lost. The team already has a stop script that finishes the current job and exits cleanly.

What is the MOST operationally efficient way to run the stop script before scale-in?

A. Add an Auto Scaling lifecycle hook, an EventBridge rule, and a Lambda function that runs the script with Run Command

B. Increase the load balancer's deregistration delay to one hour

C. Enable termination hooks on the CodeDeploy deployment group, and run the script at ApplicationStop

D. Add a shutdown script to the instances' user data

◆ PAUSE AND DECIDE

What is the MOST operationally efficient way to run the stop script before scale-in?

A Hook, rule, Lambda, Run Command

Works, but builds by hand what CodeDeploy termination hooks already provide.

B Longer deregistration delay

Only drains load balancer connections. A worker's in-process job is not a connection.

C Termination hooks, ApplicationStop

CodeDeploy runs a termination deployment before the instance is terminated, calling the existing script.

D User data shutdown script

User data runs at launch, not at termination. The script would never run at the right moment.

THE SIGNAL

Graceful scale-in for an application already deployed with CodeDeploy: termination hooks.

A web application runs on an Auto Scaling group behind an Application Load Balancer. CodeDeploy performs in-place deployments with automatic rollback on alarms.

When a bad release is detected, rollback takes around 25 minutes, and users see errors throughout. The company wants the fastest possible rollback and full capacity during deployments.

What should the DevOps engineer change?

A Switch the deployment configuration to OneAtATime

B Add more CloudWatch alarms so that rollback starts sooner

C Increase the Auto Scaling group's desired capacity during deployments

D Use a blue/green deployment that copies the Auto Scaling group, keeping the original instances for a wait period

◆ PAUSE AND DECIDE

What should the DevOps engineer change?

A OneAtATime

Limits exposure, but makes both deployment and in-place rollback slower still.

B More alarms

Detection was not the slow part. The in-place redeployment was.

C More capacity

Helps capacity, but rollback is still a full in-place redeployment.

D Blue/green, keep the original fleet

Rollback just moves the load balancer back to instances still running the old version.

THE SIGNAL

Rollback too slow on in-place: the rollback is a redeployment. Blue/green makes it a traffic switch.

Module 6 in five lines

- Instance profile fetches the bundle; service role manages Auto Scaling and load balancers.
- Stop and block-traffic scripts run from the last successful revision. ignoreApplicationStopFailures is the escape.
- Minimum healthy hosts is what stays up. Zonal configurations go one Availability Zone at a time.
- Launch hooks install on new instances; termination hooks shut down gracefully on scale-in.
- In-place rollback is a redeployment. Blue/green rollback is a traffic switch.

> ON THE EXAM
>
> Next, module 7: containers and serverless. ECS rolling, circuit breaker, native blue/green, Lambda traffic shifting, SAM and Beanstalk.